AUTOMATED GOLD TRADING SYSTEM

xAuby

Complete Guide — Architecture, Configuration, Strategies, the Market Regime Detector, Risk Management, and Operations

---

Compiled and reorganized from every project document
(README, README_DEV, CLAUDE.md, DESIGN.md, PRODUCT.md, docs/*)
Reflects the system's latest state as of July 2026
Prepared for education and internal use only

Table of Contents

C. Full Script Reference

D. Risk Disclaimer

Preface and Current System Status

This book compiles every document in the xAuby project — `README.md`, `README_DEV.md`, `CLAUDE.md`, `DESIGN.md`, `PRODUCT.md`, and everything under `docs/` — into a single reading order, translated and reorganized in English, with duplication removed and **every point where the source documents have drifted out of date corrected** to match the system's actual state, as read from the real configuration files (`bot_config.yaml`, `coin_whitelist.json`).

> **Important note on documentation drift:** several source documents (notably `README_DEV.md`, `docs/architecture.md`, `docs/trading-flow.md`, `docs/configuration.md`, `docs/tui.md`, `docs/webui.md`, `docs/telegram.md`) still describe an older system state (Binance.th spot trading, two simultaneous pairs — `XAUTUSDT` + `BTCUSDT`, `risk_pct 0.45`) — which `CLAUDE.md` itself explicitly flags as a "doc drift warning." This book treats **the real `coin_whitelist.json` and `bot_config.yaml` as ground truth**, per CLAUDE.md's own rule, and clearly distinguishes anywhere the content describes a "system capability" (e.g. multi-pair support, multi-exchange support) from "what is actually running right now" (a single live gold pair on OKX).

Current Real System State (Quick Summary)

Item	Actual value
Exchange	OKX — USDT-settled Perpetual Swap via CCXT adapter
Live trading pair	`XAUUSDT` (gold) only
Strategy	`cdc_action_zone`, Long + Short (stop-and-reverse)
Timeframe	Primary 4-hour (4h) / confirm 1-day (1d)
Leverage	1x (isolated margin, one-way position)
Risk per trade	`risk_pct = 0.02` (2% of equity)
Max allocation per trade	25%
Max daily loss	6%
Max open positions	1
Stop-loss	Disabled (`disable_stop_loss: true`) — a CDC-pure strategy that exits only on a zone flip
Partial take-profit	Closes 50% of the position at +12% profit
RegimeRouter	Off for the XAU pair (`regime_router_enabled: false`)

These figures can change according to the operator's real configuration — always check `bot_config.yaml` and `coin_whitelist.json` before relying on them for a real decision.

Part 1 — Meet xAuby

1.1 What Is xAuby

xAuby is a single-process, multi-symbol automated trading system for perpetual swap contracts on **OKX** (`api.okx.com`), connected through an exchange-neutral adapter layer called **CCXT**. The system ingests candles and live prices over both REST and WebSocket, runs each trading pair through its own strategy plugin, can optionally route a pair through a Market Regime Classifier to select a strategy automatically, places real or simulated swap orders with ATR-based stop-loss / trailing / breakeven logic, and persists everything to SQLite and JSONL files for replay, incident review, and display on a Textual dashboard. Operators drive the system through the CLI, the Textual TUI, and Telegram.

The Python package is named `xauby`, and supports Python 3.10 and above (the README targets 3.12+; the current development container runs on 3.11).

1.2 Design Philosophy

The design principles the system has followed throughout:

- **Configuration drives behavior** — the engine never hard-codes trading pairs, strategies, or chart overlays. Everything comes from configuration files.
- **The engine is strategy-agnostic** — all signal and exit logic lives in strategy plugins, not in the core engine. Adding a new strategy never requires touching engine code.
- **Configuration responsibility is split into three layers** — Engine / Strategy / Portfolio (detailed in Part 4) — so backtest and live results never diverge (parity).
- **Fail-safe by default** — the system always starts in simulation mode. Going live requires several gates to be satisfied simultaneously (see section 2.5), and an unconfirmed router is automatically forced back to simulation.
- **Per-pair isolation** — each active trading pair gets its own strategy instance, runner, mode, and handoff state, fully separated from every other pair. Portfolio capital remains intentionally shared.
- **Rollback means flipping a switch, not reverting code** — every major feature (RegimeRouter, the event bus, the GMM cross-check, etc.) sits behind a feature flag in the `architecture:` block, and can be flipped back to `false` instantly with no code changes.

1.3 Current Runtime Baseline

The table below is the real, current baseline read from `coin_whitelist.json` and `bot_config.yaml`:

Pair	Mode	Strategy	Primary TF	Confirm TF	Sides
`XAUUSDT`	live	`cdc_action_zone`	4h	1d	Long + Short

The engine runs the exchange as **OKX Perpetual Swap** (`exchange.provider: ccxt`, `ccxt_id: okx`, `market_type: swap`, `margin_mode: isolated`, `position_mode: one_way`) at **1x leverage** (`derivatives.max_leverage: 1`) with the funding guard (`max_abs_funding_rate`) and `min_liquidation_distance_pct` active. The gold slot uses the `XAUUSDT` (perpetual) symbol; backtests proxy gold to `PAXGUSDT` since it has deeper price history on Global Binance.

The single live gold pair runs CDC Action Zone as **long and short simultaneously**, in a stop-and-reverse pattern (`allowed_sides: [long, short]`, `short_live_enabled: true`), because the swap adapter advertises `swap` / `positions` / `reduce_only` capabilities in full (`xauby/api/ccxt_client.py`).

Order flow: entries place a **LIMIT** order at the ticker (`execution.order_type: limit`), and when `execution.entry_market_fallback: true`, any unfilled remainder is topped up with a **MARKET** order after a timeout (instead of being cancelled), keeping live in parity with the market-fill backtest. Exits use MARKET on urgent triggers (CDC turning red / NO_TRADE / a forced close) and LIMIT otherwise.

The current XAU pair is **CDC-pure** (`disable_stop_loss: true`), so there is no exchange-side stop — exits are zone-flip driven only, and sizing uses a fixed-fraction `position_pct` of equity rather than an SL-distance. CDC also has a one-shot partial take-profit enabled: `partial_tp_pct: 12.0`, `partial_tp_fraction: 0.5` — the engine banks half the position at +12%, marks `trade_states.partial_tp_taken`, and lets the remainder exit on CDC.

> **Router safety gate:** a pair with `mode: live` and `regime_router_enabled: true` is forced to **sim** unless it also has `regime_router_live_confirmed: true` — **never flip this without explicit per-pair operator sign-off.** NO_TRADE regimes (`PANIC_SELL`, `BEAR_BREAKDOWN`, `BEAR_TREND_STRONG`) block new buys, keep existing stop protection, tighten trailing to 1x ATR, and can force-close after the configured number of candles (`force_close_candles`).

1.4 What This Bot Does and Does Not Do

What the bot does

- Evaluates strategy signals on the configured candles
- Opens and closes real exchange positions, or simulated ones
- Keeps stop-loss protection and local risk gates active
- Writes state/events for the dashboard, replay, incident review, and Telegram operations

What the bot does not do

- Guarantee any profit
- Manage funds sitting in unrelated OKX wallets
- Make a third-party strategy plugin automatically safe, unless the sandbox check is enabled (see Part 3)
- Replace a user's own review of risk, leverage, fees, funding, and exchange rules

Warnings for new users:

1. This repository is currently configured for exactly one live market: OKX `XAUUSDT` USDT-settled perpetual swap (mapped by CCXT to `XAU/USDT:USDT`).
2. A simulation run (`python run_xauby.py --simulate`) must succeed first, before any live order is even considered.
3. Live trading requires three gates at once: the `--live` flag, `LIVE_TRADING=true`, and the active pair set to `mode: live` in `coin_whitelist.json`.
4. The bot only reads the OKX trading/swap account it's configured for — not every OKX wallet. If the dashboard shows `0.00 USDT`, check that funds are in the account used for USDT swap trading and that the API key can read/trade it.
5. **Never enable withdrawal permission on the API key.** The bot only ever needs read and trade permissions.
6. Run one engine instance per runtime root. If multiple accounts are needed, use separate `XAUBY_HOME` or `XAUBY_INSTANCE_ID` values (see Part 10).

Part 2 — Getting Started

2.1 Installation

Requirements: Python 3.12+, a configured exchange API key, an optional Telegram bot.

```
git clone https://github.com/iisara555/xAuby.git
cd xAuby

python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
pip install -e .

cp .env.example .env
python run_xauby.py --simulate
python health_check.py
```

2.2 Configuring the OKX API Key

Before live trading, edit `.env` and provide OKX credentials:

```
OKX_API_KEY=...
OKX_API_SECRET=...
OKX_API_PASSPHRASE=...
LIVE_TRADING=true
```

OKX requires all three: an API Key, Secret, and Passphrase (one of `OKX_API_PASSPHRASE`, `OKX_PASSWORD`, or `OKX_API_PASSWORD`). **Never commit the `.env` file.**

Verify the active pair and exchange configuration before going live:

```
python scripts/evaluate_okx_xau_migration.py --config configs/okx_xau_paper.yaml --whitelist configs/ok
DEFAULT_SYMBOL=XAUUSDT python health_check.py
```

2.3 Always Run Simulation First

The system's default mode is always **simulation**. No matter what gets misconfigured, the system leans toward the safe path. Switching to live requires explicit configuration per section 2.5.

Other available entry points:

Command	What it does
`python run_xauby.py --simulate` / `--live`	Starts the trading engine directly
`python launcher.py`	Opens the interactive launcher menu (engine + TUI menus)
`xauby` / `xauby --sim` / `xauby --live`	The installed console script (`xauby.cli:main`)
`xauby restart [--live]`	A controlled restart (the live path runs a preflight script)
`xauby update`	Pulls code from GitHub, then restarts
`python -m xauby.ui.textual_tui.app`	Launches the TUI standalone
`python health_check.py`	Runs a one-shot health probe
`bin/xauby`	A bash wrapper that cd's into the project root and calls `venv/bin/python`

2.4 Common Operational Commands

Recommended command to start live, after simulation and health checks have both passed:

```
env -u DEFAULT_SYMBOL XAUBY_DEFAULT_SYMBOL=XAUUSDT XAUBY_FOCUS_SYMBOL=XAUUSDT \
  SIMULATE_ONLY=false BOT_READ_ONLY=false \
  ./venv/bin/python run_xauby.py --live --pair XAUUSDT
```

A read-only browser dashboard for mobile/desktop:

```
./scripts/start_webui.sh
```

Commands used frequently during operation:

```
xauby restart        # Restart the engine; live mode runs a controlled preflight
xauby restart --live # Force a controlled restart for live mode
xauby update          # Pull origin/main from GitHub, then restart
```

`xauby update` is a shortcut for:

```
scripts/deploy_from_github.sh --restart --branch=main
```

The controlled restart path allows positions the bot is already tracking (including a matching stop-loss order), but blocks immediately on any unknown balance or untracked open order.

2.5 Checklist Before Going Live

Live trading requires every one of these conditions at once:

1. The `--live` flag when running the command

2. The `LIVE_TRADING=true` environment variable
3. `simulate_only: false` in YAML
4. That trading pair set to `mode: live` in the whitelist

Mode	How to enable	Behavior
Simulation	`--simulate`, `simulate_only: true`, or per-asset `mode: sim`	No real orders; the SimBroker ledger is used when enabled
Live	`--live` + `LIVE_TRADING=true` + `simulate_only: false` + per-asset `mode: live`	Real orders on the configured exchange
Read-only	`--read-only` or `read_only: true`	Signals only, no orders sent
Semi-auto	`trading.mode: semi_auto`	Requires Telegram confirmation before every buy
Telegram pause	`/pause`, then confirm PAUSE	A runtime block on new buys system-wide; `/resume` allows entries again

> The full, detailed pre-live checklist lives in section **11.4** at the end of this book — read it in full before ever putting real capital in.

Part 3 — System Architecture

3.1 Layered Overview

xAuby is split into 5 layers that work together:

Layer	Main components	Role
Operator	`launcher.py`, `run_xauby.py`, Textual TUI, Telegram	Entry point and human control
Core Runtime	`LiteTradingEngine`, `PairRegistry`, `SymbolContext`, `RegimeRouter`, `StrategyRunner`, Risk/PreTradeGate, Order Executor/SimBroker	Core of the trading system
Strategy Plane	Strategy plugins, indicator plugins, the strategy/portfolio resolver	Signal logic and chart rendering
Data Plane	CCXT REST client, WebSocket, LiteDB/SQLite	Market data ingestion and state storage
Observability	EventEmitter, JSONL events, State JSON, HealthMonitor	Event logging for replay/audit/display

Data flow: the launcher/CLI invokes the engine → the engine pulls pairs from the registry → a SymbolContext is created per pair → (if enabled) RegimeRouter routes it → StrategyRunner evaluates the signal → Risk/PreTradeGate is checked → a real or simulated order is sent → results are written to the DB and as events → the State JSON is updated for the TUI/WebUI to read.

3.2 What a Single Engine Tick Does

This section describes one "tick" of `LiteTradingEngine` for a single symbol. The outer `tick()` loops over all active pairs from `PairRegistry`.

Per-symbol sequence

1. Sync candles over REST
2. Check whether the WebSocket price is stale — if so, fall back to a REST ticker; otherwise use the WS snapshot
3. Read the current trade state
4. Resolve the strategy and portfolio configuration for that pair
5. If RegimeRouter is active for this pair: classify the regime, debounce, and route; otherwise use the configured pair strategy directly
6. If the regime is NO_TRADE: block new buys and tighten protection; otherwise: let `StrategyRunner.evaluate` run
7. Manage the existing position / recovery / force close (if under a NO_TRADE condition)

8. Check the macro guard before allowing a buy — if it blocks, HOLD with a guard alert
9. By state + action: `idle + BUY` → execute buy / queue semi-auto, `bought + SELL` → execute sell, `bought + HOLD` → process partial TP / trailing SL / breakeven
10. Persist trade state + events
11. Update the state JSON for the TUI

Entry path

1. Protection checks the daily trade count, daily loss %, consecutive losses, and max open positions
2. PreTradeGate checks min notional and slippage against the signal price
3. Sizing resolves the pair's portfolio configuration, applying global deploy caps
4. The order path uses the live executor or SimBroker, depending on the pair's mode
5. The stop-loss is placed on the exchange (live) or monitored/simulated locally (sim)
6. Telegram reports the filled or simulated trade, tagged with symbol and mode

Exit path

Exit triggers include: a strategy SELL signal, a one-shot partial take-profit (`partial_tp_pct` / `partial_tp_fraction`) that banks part of the position and records a partial closed trade while leaving the remainder open for the normal exit, a local SL confirmed over `sl_confirm_ticks`, an exchange SL fill detection, a RegimeRouter NO_TRADE forced close after the configured number of candles, or `max_hold_hours` / a minimal ROI table (if configured). A failed sell attempts SL restoration, with a critical Telegram alert sent if restoration fails.

Partial take-profit path

Partial TP only runs **after** the full-exit pipeline has had a chance to sell in the same tick — if a full exit also fires, the full exit wins. Otherwise, for a `bought` state with `partial_tp_taken=false`, the engine compares the mark price against the threshold set from entry:

- LONG: `mark ≥ entry × (1 + partial_tp_pct / 100)`
- SHORT: `mark ≤ entry × (1 − partial_tp_pct / 100)`

When the threshold is hit, `execute_partial_tp()` closes the configured fraction through the broker abstraction, resizes any exchange-side stop to the remaining quantity, sets `partial_tp_taken=true`, emits a `partial_tp_executed` event, and leaves the remainder under the existing strategy/CDC exit.

Strategy handoff

When RegimeRouter changes the strategy while a position is open, the design is safe by construction:

- The open position stays with its original strategy owner until it closes
- New entries are blocked or deferred until the handoff completes

- The handoff only completes once the position has closed
- The engine emits handoff events for observability

This prevents a new strategy from managing exits for a position it did not open.

NO_TRADE handling

For regimes mapped to `None`: new BUY entries are blocked, existing stop protection remains active, the trailing stop tightens to 1x ATR when available, recovery requires `regime_router.recovery_candles` tradeable candles, and a forced close can trigger after `regime_router.force_close_candles` consecutive NO_TRADE candles. Current NO_TRADE regimes are `PANIC_SELL`, `BEAR_BREAKDOWN`, and `BEAR_TREND_STRONG`.

Short execution

Short-capable strategies emit `OPEN SHORT` / `CLOSE SHORT` rather than overloading BUY/SELL. On a USDT perpetual, opening maps to SELL and closing maps to a reduce-only BUY. The stop-loss sits above entry and the fixed take-profit sits below entry. Position reconciliation uses the exchange's position endpoint (not spot balances), and blocks trading on an orphan or side-mismatched position.

3.3 PairRegistry and SymbolContext

`PairRegistry` merges data from 3 sources: (1) assets in `coin_whitelist.json` (strategy, mode, timeframe per coin), (2) `data.pairs` in YAML when a legacy/explicit list is needed, and (3) an optional `DEFAULT_SYMBOL` from the environment. With `architecture.whitelist_strict: true`, the whitelist is the sole source of truth; with `hot_reload_enabled: true`, the engine polls file mtime every `reload_interval_seconds` seconds — on change it re-initializes symbol contexts and refreshes WebSocket subscriptions.

Each active pair gets its own `SymbolContext`, containing:

- A tick snapshot from WebSocket with REST fallback
- The current strategy and any handoff target
- The current regime and its last snapshot
- Per-symbol execution mode (`sim` or `live`)
- Semi-auto pending state
- Last signal metadata for the TUI and Telegram

Pairs are isolated at the strategy/runner/context level. Portfolio capital and global guards remain intentionally shared.

3.4 Dependency Rules

Rule	Rationale
`observability` must not import `engine`	Replay and health stay engine-agnostic
The TUI is read-only (`LiteDB(readonly=True)`)	The UI must never block or mutate trade data
Secrets live only in `.env`	`bot_config.yaml` is safe to commit because it has no keys
Exactly one engine instance	`core/.engine.lock` guards against doubles
Strategy and indicator plugins ship together	Chart/legend must match actual strategy behavior
The engine stays strategy-agnostic	Signal/exit logic belongs only to plugins/config

3.5 Strategy and Indicator Plugins

This is the most important extension pattern in the system: **a strategy and its indicator plugin must always travel together.** A PR that adds a strategy without a matching indicator plugin and a `strategy_chart_indicators` mapping is considered incomplete, because the dashboard would drift from the actual strategy behavior.

Strategy plugin

Located at `xauby/strategies/<name>/strategy.py` — subclasses `xauby.strategies.base.Strategy`, decorated with `@register("<name>")`, implementing `analyze(ctx: MarketContext) -> Signal` (with optional `default_config()` / `validate_config()`). A strategy must be a pure analyst — it must **never** place orders, touch the DB, call Telegram, or call engine methods directly. The engine auto-discovers plugins via `load_strategy(name, config)`.

Indicator plugin

Located at `xauby/strategies/indicators/indicator_<name>.py` — subclasses the `Indicator` base class. `compute(df, config)` appends named columns (no ANSI codes); `display_config` defines zones, lines, metrics, labels, and colors, feeding the terminal chart and the TUI legend/checklist.

Checklist for adding a new strategy

1. Strategy plugin + `@register`
2. Indicator plugin + `@register` (or reuse a compatible one)
3. Add `bot_config.yaml -> architecture.strategy_chart_indicators.<strategy>`
4. Write tests: a strategy test, an indicator test, and chart legend coverage

> Strategies tagged `research` — including the `*_short` family and other R&D plugins — are **hard-blocked** by the engine from ever running on a live pair (`_load_strategy_for_symbol`). They are sim/backtest only, and

> must never be mapped into `regime_router.mapping` for production.

3.6 All Feature Flags (architecture.*)

Every architecture-level feature is controlled by a flag under `bot_config.yaml -> architecture:`. Rollback means flipping the flag back to `false` — no code revert required.

Flag	Purpose
`whitelist_strict`	The whitelist is the sole pair/strategy source, with fail-fast validation
`indicator_registry_enabled`	The chart computes indicators through the IndicatorRegistry instead of hard-coding
`sync_yaml_pairs_from_whitelist`	Keeps `data.pairs` synced from the whitelist automatically
`tui_indicator_registry`	The TUI checklist/legend comes from the registry adapter
`per_symbol_execution_mode`	Each pair can independently choose `sim` / `live`
`sim_broker_enabled`	Simulated orders route through SimBroker with a persistent ledger
`regime_router_enabled`	The master switch for RegimeRouter (still requires the per-pair flag too)
`regime_confidence_filter`	Suppresses a regime switch when confidence is below the threshold
`regime_statistical_crosscheck`	Enables the parallel GMM statistical regime checker (advisory only, see section 5.4)
`event_bus_enabled`	Dispatches EventEmitter events to in-process subscribers
`exchange_plugin_registry_enabled`	Deprecated/vestigial — the gateway always uses the registry now; kept for backward compatibility
`strategy_sandbox_strict`	Rejects (not just warns about) a strategy failing the static sandbox scan
`strategy_validate_strict`	Rejects a strategy whose `config_errors()` is non-empty at load time

Part 4 — System Configuration

4.1 The Three Core Configuration Files

File	Committed?	Role
`.env`	No	API key, `LIVE_TRADING`, Telegram
`.env.example`	Yes	Template for new installs
`bot_config.yaml`	Yes	Engine, Strategy, Portfolio, notifications, backtest optimizer
`coin_whitelist.json`	Yes	Active pairs, timeframes, per-pair strategy, mode, router gates

Runtime files under `core/` (DB, logs, locks, sim balances, backtest cache) are machine-specific — never commit them, and do not treat them as portable configuration unless intentionally copying a whole runtime root.

4.2 The Three-Layer Source-of-Truth Principle

Configuration is split into three responsibilities. Putting a value in the wrong layer is a recurring bug source because backtest, live, replay, and the optimizer all resolve through the same strategy/portfolio resolver — divergence breaks parity.

Layer	Owns	Must NOT own
Engine Config (`bot_config.yaml` top level)	Exchange, logging, monitoring, notifications, global risk guards, feature flags	Strategy signal/exit parameters
Strategy Config (`strategy.config.<id>` , `strategy.symbols.<SYMBOL>` , whitelist `strategy_params`)	RSI/ATR/volume, SL, trailing, breakeven, strategy timeframe, signal/exit logic	Credentials or sizing
Portfolio Config (`portfolio.position_sizing` , `portfolio.symbols.<SYMBOL>.position_sizing`)	Position sizing, allocation, capital	Indicator/signal rules

Rule: any value that can change a backtest result must live under Strategy or Portfolio config, never in an engine-only block. The engine is always strategy-agnostic.

4.3 Exchange Configuration (OKX via CCXT)

`bot_config.yaml -> exchange:` is the single source of truth for exchange selection and exchange-wide economics. The engine, backtest, replay, health probe, and operational scripts all resolve through the same helper (`xauby/runtime/exchange_config.py`), keeping simulation in parity with live.

```
exchange:
  provider: ccxt
  ccxt_id: okx
  name: okx
  quote_asset: USDT
  settle_asset: USDT
  fee_pct: 0.0005
  market_type: swap
  margin_mode: isolated
  position_mode: one_way
  api_key_env: OKX_API_KEY
  api_secret_env: OKX_API_SECRET
  base_url_env: OKX_BASE_URL
```

- **Credentials** are read from exchange-driven environment variables. OKX requires `OKX_API_KEY`, `OKX_API_SECRET`, and one of `OKX_API_PASSPHRASE` / `OKX_PASSWORD` / `OKX_API_PASSWORD`.
- **Fee precedence:** `exchange.fee_pct` → `backtest.fee_pct` → `trading.fee_pct` → `0.001`; a per-symbol `sim_fee_pct` (whitelist) wins for that pair.
- CCXT is the active adapter used for OKX.

4.4 Whitelist Schema and the Current Trading Pair

```
{
  "version": 1,
  "quote_asset": "USDT",
  "assets": [
    {
      "symbol": "XAU",
      "enabled": true,
      "primary_timeframe": "4h",
      "confirm_timeframe": "1d",
      "strategy": "cdc_action_zone",
      "mode": "live",
      "allowed_sides": ["long", "short"],
      "leverage": 1.0,
      "short_live_enabled": true,
      "regime_router_enabled": false,
      "regime_router_live_confirmed": false,
      "sim_fee_pct": 0.05
    }
  ]
}
```

Field	Values	Notes
`symbol`	base asset such as `XAU`	Normalizes to `XAUUSDT` via `quote_asset`
`enabled`	bool	Disabled assets are ignored
`primary_timeframe`	e.g. `1h`, `4h`	Used by live, backtest, and chart
`confirm_timeframe`	e.g. `1d` or empty	Strategy dependent
`strategy`	plugin id	Required when `whitelist_strict: true`
`strategy_params`	object	Per-pair overrides merged into strategy config
`mode`	`sim`, `live`	Requires `per_symbol_execution_mode: true`
`regime_router_enabled`	bool	Pair-level RegimeRouter gate
`regime_router_live_confirmed`	bool	Required before a live pair may route strategies
`sim_fee_pct`	number	Percent, e.g. `0.05` = 0.05%

4.5 Environment Variables

Variable	Default	Description
`OKX_API_KEY / OKX_API_SECRET / OKX_API_PASSPHRASE`	-	OKX REST/WS auth
`LIVE_TRADING`	`false`	An additional gate for real orders
`SIMULATE_ONLY`	from YAML	CLI `--live` / `--simulate` override
`BOT_READ_ONLY`	`false`	Skips order placement
`TELEGRAM_ENABLED`	`false`	Master switch for alerts
`TELEGRAM_BOT_TOKEN`	-	Bot token from @BotFather
`TELEGRAM_CHAT_ID`	-	The authorized chat for commands
`DEFAULT_SYMBOL`	-	Optional single-pair override
`XAUBY_HOME`	`core`	Base directory for runtime data — relocate to run instances side by side (see Part 10)
`XAUBY_INSTANCE_ID`	-	Optional sub-namespace under the runtime root

Regime classifier macro weights

The market regime classifier combines DXY, interest rates, and news into a macro bias. Weights are configurable so the classifier isn't hardwired to gold:

```
macro_sentiment_guard:
  macro_weights:   # defaults shown (gold thesis)
    news: 1.0
    dxy: 1.0       # already oriented at the producer (positive = bullish); must not be re-inverted
    fred: 1.0
  regime_bias_threshold: 0.3   # |combined_macro| above this flips to RISK-ON/RISK-OFF
```

4.6 The Built-in Quick Config Menu

Launcher menu → **Quick Configuration** (or `xauby --config`) opens a native Textual config editor: a grouped `OptionList` hub whose categories push submenus of toggles, number+range modals, and pickers (arrow/Enter/mouse; secrets are masked). Writes go through a `ruamel.yaml` round-trip, so **comments and structure in `bot_config.yaml` are preserved** across edits.

Group	Edits
Trading & Risk	Mode (sim/live), per-pair risk percentage, SL ATR, breakeven, strategy
Global Trading	`max_open_positions`, `interval_seconds`, timeframe, min order amount
Risk Guards	Drawdown guard, max daily loss %, max consecutive losses
Portfolio	Initial balance, per-symbol allocation %
Regime Router	Enable, confidence threshold, debounce/recovery/force-close candles, regime → strategy mapping
Telegram Alerts & Schedule	Credentials/test, weekly/daily digest scheduling
Exchange	A guided switch-exchange wizard, per-exchange API credentials, a connection test

5.1 Every Strategy Plugin in the System

The system ships 15 strategy plugins total. Some run in production, others are research-only (sim/backtest):

Strategy	Indicator plugin	Chart zones	Chart lines
`cdc_action_zone` ⭐Live	`cdc_action_zone`	Blue/Green/Red/Neutral	EMA12, EMA26
`supertrend_ema200`	`supertrend`, `ema200`	ST Bull/Bear/Neutral	SuperTrend, EMA200
`bbkc_squeeze`	`bbkc_squeeze`	Compressed/Breakout/Neutral	Bollinger Bands, Keltner Channels
`bbrsi_mean_reversion`	`bbrsi_mean_reversion`	Oversold/Neutral/Overbought	Bollinger Bands
`btc_ema_pullback`	`btc_ema_pullback`	Trend/Pullback/Reclaim/Neutral	EMA Fast/Slow/Trend
`ict_lite_strategy`	`ict_lite`	Sweep Low/Reclaim/MSS/Neutral	EMA Fast/Slow, recent High/Low
`rsi2_meanrev` (research)	`rsi2_meanrev`	Buy Setup/Oversold/Exit/Neutral	EMA200, SMA5
`sol_ema_pullback`	`sol_ema_pullback`	Trend/Pullback/Reclaim/Neutral	EMA20, EMA50
`vol_breakout` (research)	`vol_breakout`	Breakout/ATR Expansion/Neutral	Range High, Exit EMA
`xauby_vwap_pullback`	`xauby_vwap_pullback`	Entry/Pullback/Trend/Wait/Trend lost	EMA20/50/200, VWAP
`donchian_trend`	-	-	Turtle breakout above EMA200 + ADX filter
`donchian_short` / `supertrend_short` / `rsi2_short` (research)	-	Short-side, research only — never in production	

⭐ = the strategy currently running on the live pair (XAUUSDT)

5.2 CDC Action Zone — the Live Strategy in Production

`cdc_action_zone` is the only strategy currently trading live. It runs as a **stop-and-reverse**: opening Long or Short as the zone signal flips. There is no exchange-side stop-loss (`disable_stop_loss: true`) — every exit is zone-flip driven. Position size is computed from a fixed `position_pct` of equity (not an SL distance), and it has a one-shot partial take-profit at +12% banking 50% of the position.

Parameter (from the real whitelist)	Value
`sl_atr_mult`	2.0
`trailing_atr_mult`	1.8
`breakeven_sl_enabled`	false
`require_fresh_zone`	true (fresh_zone_window: 1)
`require_slow_slope`	true (slow_slope_bars: 3)
`position_pct`	0.95
`partial_tp_pct` / `partial_tp_fraction`	12.0 / 0.5
`backtest_data_proxy`	`PAXGUSDT` (used as a gold proxy for backtesting)

5.3 What the Market Regime Detector Is and How It Works

The Market Regime Detector (`xauby/regime/classifier.py` , function `classify_market()`) is a rule-based classifier that analyzes the market across 4 axes and synthesizes it into one of 11 "regimes," loosely following Wyckoff phase logic (accumulation → markup → distribution → markdown):

The 4 axes analyzed

- **Trend** — the EMA12/EMA26/EMA200 stack, distance of price from EMA200, and 5/20-bar momentum
- **Volatility** — ATR relative to price, then ranked against its own **120-bar historical average** (relative, not absolute — the key detail that makes it generalize across assets and eras)
- **Liquidity** — volume ratio against a 20-bar SMA
- **Macro** — combines DXY + interest rates (FRED) + news sentiment, clamping each input to [-1, 1] before combining (preventing any one input — such as an unclamped AI news score — from dominating the result)

All 11 regime labels

Regime	Meaning	Phase
`BULL_BREAKOUT`	An upside breakout with volume confirmation and expanding volatility	MARKUP
`BULL_TREND_STRONG`	A strong uptrend with high momentum/EMA spread	MARKUP
`BULL_TREND_WEAK`	A weak uptrend, or macro conflicting with the trend	MARKUP/RECOVERY_RALLY
`BEAR_BREAKDOWN`	A downside breakdown with expanding volatility	MARKDOWN
`BEAR_TREND_STRONG`	A strong downtrend	MARKDOWN
`BEAR_TREND_WEAK`	A weak downtrend	DISTRIBUTION
`PANIC_SELL`	A panic sell-off — a downtrend with a volatility spike and severely negative macro or momentum	CAPITULATION
`LOW_VOL_ACCUMULATION`	Low volatility, price hugging EMA200 — an accumulation phase	ACCUMULATION
`LOW_VOL_RANGE`	Low volatility, but not close enough to EMA200	ACCUMULATION
`VOLATILITY_EXPANSION`	Expanding volatility with no clear trend, plus a wide range	TRANSITION
`SIDEWAYS_CHOP`	Matches none of the above conditions — directionless chop	RANGE

Confidence score

Every regime comes with a confidence score (0.05-0.98) computed as a composite of several factors: trend quality (34%), macro alignment (20%), macro conviction (14%), volatility clarity (14%), liquidity confirmation (10%), regime stability (8%) — with penalties when macro conflicts with the trend or transition risk is high. This is not a simple 0/1 gate like a basic system would use.

Guarding against insufficient-data errors

The engine always feeds the classifier 250 bars (more than the 200 EMA200 requires), but the classifier itself has an internal guard: if a fallback-computed EMA200 is derived from fewer than 200 actual bars (e.g. when called with a short window elsewhere), the system refuses to assert a Bullish/Bearish trend and forces NEUTRAL with a reduced confidence score — protecting against a "fake EMA200" from insufficient data producing a false trend signal.

5.4 The Statistical Regime Cross-Check (GMM)

To go beyond the rule-based classifier (whose thresholds are hand-tuned), the system includes a purely **statistical** regime detector (`xauby/regime/statistical.py`) that runs in parallel as a "second opinion" carrying no bias from hand-set thresholds:

- Fits a **Gaussian Mixture Model (GMM) with diagonal covariance via EM** over a 2-dimensional feature space: trend (rolling log-return sum) and log realized volatility
- These two axes were chosen because real-data validation proved the regime separates volatility far more clearly than direction
- Returns the cluster of the latest bar, with a **posterior probability** and **empirical persistence (stay probability)**
- Implemented in **pure numpy** with no scipy/sklearn, keeping the main `classifier.py` pure-stdlib

The result is attached as `stat_*` features on the regular regime output, **advisory only — it never overrides the regime or routing**. Off by default behind the `architecture.regime_statistical_crosscheck` flag, tunable via the `regime_statistical:` block (`components`, `min_bars`).

5.5 Regime Detector Accuracy Report

"Accuracy" for a regime detector is **not** "percent correct" — there is no ground truth for which regime the market was "really" in at any given moment. The correct measures are three:

1. **Volatility separation** — does the regime actually separate realized volatility? (The primary job, since it drives position sizing and stop width.)
2. **Persistence** — does the regime last long enough to act on, or does it churn every bar?
3. **Directional tilt** — do bull-family regimes actually out-drift bear-family regimes? (Expected to be *small* always — a bonus, not the headline metric.)

Measured with **eta² (the fraction of return variance explained by the regime label)** along with a permutation p-value (the probability a random relabeling would separate the data equally well), on real 4-hour data (PAXGUSDT as a gold proxy, ~12,800 bars / BTCUSDT ~17,500 bars):

Metric	PAXGUSDT (gold proxy)	BTCUSDT (cross-check)
Volatility separation eta² (p)	**0.041** (p = 0.0005)	**0.034** (p = 0.0005)
Directional separation eta² (p)	0.0004 (p = 0.85 — noise)	0.0019 (p = 0.0015)
Persistence (mean run / flip rate)	6.6 bars / 15.1%	5.2 bars / 19.2%
Drift ordering (bull vs. bear)	+1.3 vs −0.0 bps ✓	+5.8 vs +0.5 bps ✓
GMM cross-check agreement	50.0%	45.2%

Conclusion: the classifier does its primary job well — volatility separation is strong and statistically significant ($p \le 0.001$ on both assets), regimes are stable enough to trade (mean run > 3 bars, meaning the system's 3-candle debounce is well tuned), and the bull/bear drift ordering is correct. Directional edge is weak, as expected (normal for any classifier in real markets). **Practical conclusion: use the regime to size positions by volatility, not to predict the direction of the next bar.**

> The GMM cross-check agrees with the rule-based classifier only about 45-50% of the time, and **this is by design**. If the cross-check agreed nearly 100% of the time, it wouldn't be a genuine second opinion. Disagreement signals that the current read is ambiguous — valuable information for an operator or a future confidence gate, not evidence that either system is wrong.

Limitations to know

- eta^2 is small in absolute terms (~0.04) — markets are mostly noise, and no regime model explains a large share of bar-to-bar variance. 0.04 with $p \leq 0.001$ means the separation is real and robust, not that it's large.
- Gold history is proxied via PAXGUSDT, since the live pair XAUUSDT itself has a shorter history.
- Thresholds are still hand-tuned, not fit per asset. The GMM cross-check is the first step toward a more data-driven model.
- Forward returns are in-sample descriptive statistics, not a tradeable backtest proving any particular strategy is profitable.

5.6 RegimeRouter — Automatic Strategy Selection

RegimeRouter maps each regime to the strategy suited to it. It is opt-in per pair:

1. The global `architecture.regime_router_enabled` flag must be true
2. The per-asset `regime_router_enabled` flag must be true
3. A live asset also needs `regime_router_live_confirmed: true`

If a live asset enables the router without live confirmation, the engine forces it to sim. Other router behavior: a 3-candle debounce before confirming a regime switch, NO_TRADE blocks new buys while keeping existing stop protection, recovery requires 3 tradeable candles before resuming, a forced close after 6 consecutive NO_TRADE candles, and a safe strategy handoff as described in section 3.2.

> **Current real status:** the XAU pair has RegimeRouter off (`regime_router_enabled: false`) — meaning the system currently runs `cdc_action_zone` directly, without automatic strategy selection. The table below is the mapping configured in the system, ready to use once the flag is enabled in the future:

Regime	Mapped strategy	Action
`BULL_BREAKOUT`	`cdc_action_zone`	BUY allowed
`BULL_TREND_STRONG`	`cdc_action_zone`	BUY allowed
`BULL_TREND_WEAK`	`cdc_action_zone`	BUY allowed
`LOW_VOL_ACCUMULATION`	`bbrsi_mean_reversion`	BUY allowed
`LOW_VOL_RANGE`	`bbrsi_mean_reversion`	BUY allowed
`VOLATILITY_EXPANSION`	`supertrend_ema200`	BUY allowed
`SIDEWAYS_CHOP`	`bbrsi_mean_reversion`	BUY allowed
`BEAR_TREND_WEAK`	`cdc_action_zone`	BUY allowed
`PANIC_SELL`	`None`	NO_TRADE
`BEAR_BREAKDOWN`	`None`	NO_TRADE
`BEAR_TREND_STRONG`	`None`	NO_TRADE

When the backtest regime filter is enabled, it mirrors this routing: BUY signals are suppressed whenever the classifier's regime falls outside the active strategy's target regime set, keeping replay results close to the live entry gate.

5.7 Data-Driven Strategy Selection (R&D)

The system has a data-driven gatekeeper for changing a strategy in `regime_router.mapping`: a candidate may replace an incumbent **only if** it beats the incumbent's Profit Factor on both the train (oldest 70%) and test (newest 30%) splits of multi-year real market data, with enough trades and bounded drawdown. **The data decides, not belief.**

Decision criteria (all must pass)

1. Candidate PF > incumbent PF, on both train and test
2. Trade count ≥ 20 (train) and ≥ 8 (test)
3. PF_test ≥ 1.0 and maxDD_test ≤ 1.5x the incumbent's (and ≤ 15% absolute)
4. The most recent ~6-month window: PF ≥ 0.8 (a sanity check against flipping in current conditions)

Failing any one criterion keeps the incumbent for that mapping key.

> **Latest verdict:** across 26,297 real bars of BTCUSDT, no candidate passed every criterion — `donchian_trend` won clearly on train (PF 1.16 vs. 0.77) but broke down on test (PF 0.32, below both the 1.0 threshold and the incumbent's PF), so **the incumbent strategy stays in place** across every regime family. This test was run on BTCUSDT, which is not currently a live pair — it serves as a methodology example for future strategy change decisions.

Related commands: `scripts/fetch_global_klines.py`, `scripts/regime_strategy_eval.py --grid`, `scripts/regime_churn_analysis.py` (measures regime switching frequency and its impact on live strategy execution).

Part 6 — Risk Management

6.1 Position Sizing

Sizing is risk-based and auto-compounding:

```
qty = (equity × risk_pct) / sl_distance
```

Capped by `max_position_per_trade_pct` and per-asset allocation (`allocation_pct`). Equity includes open positions at mark price, so profits grow the next position size automatically (auto-compounding).

> **Important:** `risk_pct` must be kept aligned between `trading.risk_pct` (live reads from `portfolio.position_sizing.risk_pct`) and `portfolio.position_sizing.risk_pct` (backtest reads from `trading.risk_pct`) — a mismatch breaks live/backtest parity. The system's current value is `0.02` (2%) at both locations. A startup guard (`MAX_SANE_RISK_PCT = 0.10`) rejects any `risk_pct` above 10% immediately.

6.2 Stop-Loss / Trailing / Breakeven

Exit mechanisms the system supports (not every strategy uses all of them — see the real CDC values in section 5.2):

- **Initial ATR stop-loss** — an initial distance from entry, multiplied by `sl_atr_mult`
- **Trailing stop** — moves with ATR × `trailing_atr_mult` as price moves favorably
- **Breakeven** — moves the SL to the entry point once profit reaches the configured threshold (if enabled)
- **Multi-tick local SL confirmation** — for strategies without an exchange-side SL, the signal must be confirmed over `sl_confirm_ticks` before actually closing, to guard against a brief price spike

6.3 Partial Take-Profit

Strategy config can enable a one-shot partial take-profit:

```
strategy:
  params:
    cdc_action_zone:
      partial_tp_pct: 12.0
      partial_tp_fraction: 0.5
```

When an open position reaches `partial_tp_pct` return from entry, the engine closes `partial_tp_fraction` of the remaining position with a reduce-only close on live swaps (or the SimBroker in sim), records a partial closed trade, sets `trade_states.partial_tp_taken=1`, and leaves the rest to the normal strategy exit. The trigger is one-shot per position and persists across restarts. For the current XAU

live baseline this banks 50% at +12% and lets the remainder ride until CDC exits. UI surfaces show the target as `PTP`, `Partial TP`, or `PTP banked` once taken.

6.4 Drawdown Guard — the Portfolio-Level Kill-Switch

`risk.drawdown_guard` is a portfolio kill-switch: it blocks new BUYs (and force-closes positions if `close_positions: true`) when equity falls `max_drawdown_pct` below its persisted high-water mark (`core/equity_peak.json`).

6.5 Short Execution and Its Safety Conditions

The current pair enables short trading live (`allowed_sides: [long, short]`, `short_live_enabled: true`) because the OKX swap adapter advertises `swap` / `positions` / `reduce_only` capabilities in full. Safety conditions for short trading:

- Must be explicitly allowed per pair via `allowed_sides`
- Live also requires `short_live_enabled` — otherwise it's paper-only
- Startup fails closed if the selected adapter doesn't expose swap/positions/reduce-only capabilities
- Leverage defaults to 1x, capped at a maximum of 3x
- If the streaming channel (CCXT Pro) degrades, new shorts are blocked immediately; REST fallback never changes exchange or market type

6.6 Risk Alignment Checklist

Worth checking these are consistent before going live:

Key	Current value for XAU
`trading.risk_pct`	0.02
`portfolio.position_sizing.risk_pct`	0.02
`trading.max_position_per_trade_pct`	25.0
`trading.max_daily_loss_pct`	6.0
`trading.max_open_positions`	1
`derivatives.max_leverage`	1

Global guards apply across all symbols. Per-symbol strategy risk state is isolated, but portfolio capital is shared by design.

7.1 Backtest Data Sources

Backtesting pulls from **Global Binance** (`api.binance.com`) by default, for longer history (`backtest.data_base_url` , overridable via the `BINANCE_BACKTEST_URL` env var); live trading uses the configured OKX endpoint. Backtest and live resolve config through the same resolver, keeping backtest, replay, and live trading aligned with a single source of truth.

7.2 Backtest and Optimizer Commands

```
python scripts/replay_backtest.py --symbol XAUUSDT --config bot_config.yaml
python scripts/optimize_pair_configs.py
```

Strategy selection is a dry-run by default:

```
python scripts/select_pair_strategy.py --symbol BTCUSDT --candidates a,b,c
python scripts/select_pair_strategy.py --symbol BTCUSDT --candidates a,b,c --apply
```

Only `--apply` writes the best passing candidate back to YAML.

7.3 Regime-Aware Replay

Setting `use_regime_filter: true` on a symbol's strategy config activates the regime classifier inside the replay engine — BUY signals are suppressed outside the strategy's `TARGET_REGIMES` , re-evaluated every `regime_update_bars` bars (default 24).

7.4 Replay Validation After Restart/Incident

Use this after every restart, incident, or configuration change:

```
python scripts/replay_validate.py <run_id> --symbol XAUUSDT
```

Replay validation checks whether recorded live `signal_evaluated` events match the strategy replay for the same run — validating strategy output before macro guard, cooldown, and execution overrides.

8.1 Textual TUI (Terminal Dashboard)

xAuby ships a **Textual** terminal UI that reads live engine state. It does not run the trading loop itself, and it opens the database in read-only mode.

Screen	Keys	Purpose
Dashboard	`1`, `d`	Charts, pair table, regime, position, strategy/mode badges
Trade log	`2`, `t`	Closed trades and history
Incident explorer	`4`, `i`	Run timelines from the event store

Global keys: `[`, `0` previous symbol; `]`, `9` next symbol; `q` quit.

Charts and legends follow the selected pair's active strategy (source of truth per section 3.5). The UI adapts to terminal width: ≥110 columns is two columns (chart left, stats right); 75-109 is a single full-width column; <75 is compact phone-style rows with a shorter chart.

When a position is open and the config includes `partial_tp_pct`, the positions panel shows a `PTP` row with the fraction to bank, the trigger price, and its `pending` / `banked` status.

Recommended via tmux on a VPS:

```
./scripts/start_dashboard_tmux.sh
./scripts/attach_dashboard_tmux.sh
```

8.2 Mobile WebUI (Read-Only)

The WebUI is a mobile-first, read-only browser dashboard — **it never places, cancels, or modifies any trade.**

```
./scripts/start_webui.sh
```

Default bind: `127.0.0.1:8787`

The home view shows live OKX runtime state, the last 24 OHLC candles, EMA12/EMA26 overlays, CDC Action Zone detail, and partial-TP status for any open position.

Accessing it from Windows

```
ssh -L 8787:127.0.0.1:8787 user@your-vps-ip
```

Then open `http://localhost:8787`.

Accessing it from a phone

The easiest free option is Tailscale: install it on the VPS and the phone, log into the same tailnet, and reach the WebUI over the VPS's Tailscale address.

> Never bind the WebUI to `0.0.0.0` on a public VPS unless proper authentication is added.

All endpoints are read-only: `/api/state`, `/api/health`, `/api/recent-events`, `/api/trades`, `/api/candles`.

8.3 UI Design Philosophy

The WebUI is designed as a "private gold desk" — the sole owner checks it from an iPhone (Safari, often via Tailscale) or a desktop. The context is a quick glance, not a work session — the single question that must be answered within 10 seconds is: "Is the bot alive, what's my equity, what position am I in, did anything happen?"

Design principles

- **Glanceable first** — hierarchy: equity → position → signal → everything else, with no scrolling required
- **Semantic color only** — green/red/amber mean profit/loss/warning and nothing else; the brand's gold accent marks identity/selection only, never state
- **Read-only means calm** — no affordance implies control; anomalies get loud, normal operation stays quiet
- **Phone-native, Safari-first** — designed for iOS Safari's realities (dynamic toolbar, safe areas, touch targets ≥44px); desktop gets the same phone layout, centered — not a different one
- **Numbers are the interface** — tabular figures, consistent precision, aligned columns; typography does the work decoration used to do

Color palette (OKLCH)

Token	Role
`--bg-0` / `--surface`	Page background / cards-panels (dark only — `color-scheme: dark`)
`--gold`	The single brand accent — identity/selection only
`--pos` / `--neg` / `--warn`	Profit/long — loss/short — caution/degraded
`--info`	Supporting info, e.g. EMA26, regime info

Typography uses a single iOS system font stack (no webfonts); all figures use `font-variant-numeric: tabular-nums`. Phone-first layout at `min(430px, 100%)` width, with three views (Home/Signal/Activity) toggled by `body[data-view]`.

Part 9 — Telegram

9.1 Setting Up the Telegram Bot

1. Create a bot via @BotFather
2. Get your chat id, for example via @userinfobot
3. Configure `.env`:

```
TELEGRAM_ENABLED=true
TELEGRAM_BOT_TOKEN=...
TELEGRAM_CHAT_ID=...
```

Enable polling in `bot_config.yaml`:

```
notifications:
  alert_channel: telegram
  telegram_command_polling_enabled: true
```

9.2 All Commands

Only the configured `TELEGRAM_CHAT_ID` is authorized:

Command	Description
`/help`, `/start`	Command list
`/status`	All pairs: price, mode, position, last signal
`/pnl`	7-day and 30-day portfolio PnL, plus per pair
`/regime`	Regime snapshot per pair
`/last`	Recent closed trades per pair
`/health`	Engine health snapshot: mode, pause state, pairs, open positions, feed/WS issues
`/pause`	Confirms an emergency pause; blocks new BUYs without closing positions
`/resume`	Confirms resume; allows new BUYs again

9.3 Automatic Alerts

Event	Level
Engine started / stopped	trade / info
BUY/SELL filled	trade
Macro guard blocked a BUY	info
Regime score/label change	info
RegimeRouter handoff / NO_TRADE	info / position
Trailing stop updated	position
SL restore failure	position / critical
Daily digest / weekly review	info

9.4 Semi-Auto Mode and Emergency Pause

```
trading:
  mode: semi_auto
notifications:
  semi_auto_confirm_timeout_seconds: 60
```

The bot sends an inline keyboard (`Confirm BUY` / `Skip`) before every entry.

`/pause` and `/resume` are two-step commands — the text command only asks for confirmation, and only the inline button applies the runtime control. Pause blocks new buys across every path (signal, manual, semi-auto) without closing existing positions. Critical alerts always come with `Status` and `Ack` buttons.

10.1 Running Multiple Instances on One Host

xAuby separates config from mutable data, so several engines (different accounts / exchanges / pair sets) can run side by side from the same code checkout without colliding:

Concern	Env var	Default
Config (`bot_config.yaml` , whitelist, `.env`)	`XAUBY_CONFIG_DIR`	Current working directory
Mutable data (DB, lock, state, logs, sim balances)	`XAUBY_HOME` + `XAUBY_INSTANCE_ID`	`core/` (+ `/<id>`)

```
XAUBY_CONFIG_DIR=instances/acct1 XAUBY_INSTANCE_ID=acct1 \
  python run_xauby.py --live
```

Example systemd template for multiple instances:

```
[Service]
Environment=XAUBY_CONFIG_DIR=/root/xAuby/instances/%i
Environment=XAUBY_INSTANCE_ID=%i
WorkingDirectory=/root/xAuby
ExecStart=/root/xAuby/venv/bin/python run_xauby.py --live
```

Enable with `systemctl enable --now xauby@acct1` . Each instance reads its own `.env` , so secrets never need to be shared between accounts.

10.2 Multi-Exchange via CCXT

Switch exchange entirely through the launcher menu, no file editing required: option 4 → Exchange & Flags → the guided wizard picks a provider, `ccxt_id` , `quote_asset` , prompts for credentials (writing the correct `.env` var for that exchange), runs a connection test, and offers a restart.

```
exchange:
  provider: ccxt
  ccxt_id: kraken
  api_key_env: KRAKEN_API_KEY
  api_secret_env: KRAKEN_API_SECRET
  quote_asset: USDT
```

Exchange plugin registry + WebSocket streaming

Enable with the flag:

```
architecture:
  exchange_plugin_registry_enabled: true
```

With it on: `provider: binance` → the native client + BinanceWebSocket; `provider: ccxt` (or any ccxt_id) → CCXTExchangeClient + CCXTProWebSocket (real streaming for any ccxt.pro-supported exchange).

Adding a new exchange (drop-in, no factory edits)

Create a single module `xauby/api/exchanges/exchange_<name>.py` that registers both halves of the bundle via the `@register_exchange` and `@register_ws` decorators — modules are auto-discovered by `pkgutil` (filename prefix `exchange_`).

10.3 OKX Perpetuals and Short-Trading Safety

The certified derivatives path uses `market_type: swap`, `margin_mode: isolated`, `position_mode: one_way`, and a USDT settle asset. Startup fails closed when the selected adapter does not expose swap, positions, and reduce-only capabilities. CCXT Pro subscribes to ticker, OHLCV, public trades, and order book. A degraded channel blocks new shorts; REST fallback never changes exchange or market type.

Part 11 — Operations

11.1 Deploying on a VPS

```
mkdir -p core/logs
env -u DEFAULT_SYMBOL XAUBY_DEFAULT_SYMBOL=XAUUSDT XAUBY_FOCUS_SYMBOL=XAUUSDT \
  SIMULATE_ONLY=false BOT_READ_ONLY=false \
  ./venv/bin/python run_xauby.py --live --pair XAUUSDT

./scripts/start_dashboard_tmux.sh
./scripts/attach_dashboard_tmux.sh
```

Use exactly one engine instance **per runtime root**. `<root>/.engine.lock` prevents duplicates within the same root.

11.2 Update and Restart Commands

For normal VPS operations once the CLI is installed, prefer:

```
xauby update   # deploy origin/main + controlled restart
xauby restart  # controlled restart without pulling code
```

Manual fallback:

```
scripts/deploy_from_github.sh --restart --branch=main
scripts/controlled_restart_engine.sh
```

11.3 Checking VPS Latency

Use this when the bot feels slow, the TUI lags, or exchange requests time out:

```
./venv/bin/python scripts/vps_latency_check.py
./venv/bin/python scripts/vps_latency_check.py --json
```

Value	Meaning
curl total	Rough HTTPS request time to the exchange API
curl ttfb	Time to first byte; high values usually mean routing/API latency
tcp443	Raw TCP connect time; high values point to network distance/routing
clock	`NTPSynchronized=yes` matters because signed requests use timestamps
loadavg	Sustained load above available CPU cores can make the bot/TUI lag
disk_used / free	Low free space or a slow disk can hurt DB/event/state writes

The TUI header surfaces per-step timing: REST latency, WS tick age, whole engine tick duration, candle sync duration, and state JSON export duration. If REST is high but the overall tick is low, the VPS routing/exchange path is likely the issue; if the tick/sync/state timings themselves are high, reduce local work or polling/logging frequency.

11.4 The Full Safety Checklist

- ☐ Simulation run reviewed before live
- ☐ The OKX API key has trade permission for the intended account and **no** withdraw permission
- ☐ OKX API key, secret, and passphrase env vars are present before a live restart
- ☐ Funds are in the OKX account the bot reads for USDT swap trading; the dashboard can show 0.00 if funds sit in an unrelated wallet
- ☐ `risk_pct`, `max_position_per_trade_pct`, and global guards reviewed for account size
- ☐ `risk.drawdown_guard` threshold set for your risk tolerance (the kill-switch on portfolio drawdown)
- ☐ The OKX XAU long/short migration evaluation reviewed after downloading fresh 4H candles and funding history
- ☐ `regime_router_live_confirmed: true` set only after explicit per-pair sign-off
- ☐ `architecture.strategy_sandbox_strict: true` when running third-party/untrusted strategy plugins
- ☐ `scripts/select_pair_strategy.py` report reviewed before using `--apply`
- ☐ `scripts/replay_validate.py <run_id> --symbol <PAIR>` passes after a restart
- ☐ Telegram tested; `/status` shows all active pairs and `/health` shows no unexpected issues
- ☐ Emergency Telegram `/pause` and `/resume` confirmation buttons tested in simulation
- ☐ The engine is supervised by tmux/systemd, not an unmanaged SSH shell

Appendix

A. Glossary

Term	Meaning
ATR	Average True Range — the average per-bar price volatility, used to set SL/trailing distance
EMA	Exponential Moving Average — a moving average weighted toward recent data
PF (Profit Factor)	Total profit divided by total loss — above 1 means net profitable
DD (Drawdown)	How far equity has fallen from its highest point so far
Regime	The market condition label assigned by the classifier, e.g. BULL_TREND_STRONG, PANIC_SELL
Debounce	Waiting for a signal to repeat over several bars before confirming a real change (noise guard)
NO_TRADE	A regime the system treats as "don't open new positions," while still managing existing ones
Handoff	Safely transferring position management from one strategy to another
Partial TP	Closing part of a position once profit hits a set threshold (one-shot)
Reduce-only	An order guaranteed only to shrink a position, never to open a new one
Sandbox strategy scan	A static AST scan that rejects plugins touching the engine/DB/network or using eval/exec
GMM (Gaussian Mixture Model)	A statistical model that clusters data as a mixture of several normal distributions
eta^2 (eta-squared)	The fraction of variance in the data explained by a grouping variable (here, the regime label)
Permutation p-value	The probability that a random relabeling would separate the data just as well — low means high significance

B. Project Layout

```
xAuby/
|-- run_xauby.py
|-- launcher.py            # thin shim re-exporting the xauby.launcher package
|-- bot_config.yaml
|-- coin_whitelist.json
|-- docs/
|-- scripts/
|-- tests/
`-- xauby/
    |-- launcher/          # interactive launcher: config_io, process, maintenance, quick_config
    |-- engine/            # LiteTradingEngine (mixins), risk, orders, regime routing
    |-- strategies/        # strategy plugins + indicators/
    |-- runtime/           # pair registry, whitelist validation, config resolvers
    |-- regime/            # regime classifier + GMM cross-check
    |-- backtest/          # replay engine, optimizer, metrics
    |-- observability/     # EventEmitter, JSONL store, replay validation
    `-- ui/                # terminal chart + Textual TUI, dashboard, tradelog
```

C. Full Script Reference

Command	Purpose
`python -m unittest discover -s tests -q`	Runs the full test suite
`python health_check.py`	One-shot health probe
`python scripts/incident_explorer.py list`	Lists recorded incidents
`python scripts/replay_validate.py <run_id> --symbol XAUUSDT`	Validates replay after a restart
`python scripts/evaluate_okx_xau_migration.py`	Evaluates the OKX XAU migration
`python scripts/select_pair_strategy.py`	Strategy selection (dry-run/--apply)
`python scripts/optimize_pair_configs.py`	Low-CPU optimizer
`python scripts/capture_tui_screenshots.py`	Regenerates TUI screenshots
`python scripts/fetch_global_klines.py`	Downloads multi-year candles from Global Binance
`python scripts/regime_strategy_eval.py --grid`	Evaluates strategies per regime as a gatekeeper
`python scripts/regime_forward_validate.py`	Measures regime detector accuracy on real data
`python scripts/regime_churn_analysis.py`	Measures regime switching frequency
`python scripts/vps_latency_check.py`	Checks VPS latency

D. Risk Disclaimer

> This software is for education and research. Cryptocurrency and derivatives trading involves substantial risk of capital loss. Past performance does not guarantee future results. Users must comply with the configured exchange's API terms and applicable laws and regulations in their own jurisdiction at all times. The authors of this document accept no responsibility for any losses arising from using this software with real funds.

— End of book —